



OAuth 2.0 Authentication

I. Giới thiệu về OAuth 2.0

OAuth 2.0 là một framework xác thực và ủy quyền phổ biến, cho phép các ứng dụng web và di động yêu cầu quyền truy cập hạn chế vào dữ liệu của người dùng từ các dịch vụ khác mà không cần tiết lộ thông tin đăng nhập của người dùng.

II. Cấu trúc của OAuth 2.0

OAuth 2.0 có ba bên chính:

- **Ứng dụng khách (Client Application):** Ứng dụng mà người dùng muốn sử dụng và cần truy cập vào dữ liệu người dùng.
- **Chủ sở hữu tài nguyên (Resource Owner):** Người dùng sở hữu dữ liệu mà ứng dụng khách muốn truy cập.
- **Nhà cung cấp dịch vụ OAuth (OAuth Service Provider):** Dịch vụ kiểm soát dữ liệu người dùng và cung cấp API.

III. Quy trình hoạt động của OAuth 2.0

Quy trình OAuth 2.0 diễn ra qua các bước cơ bản sau:

1. **Yêu cầu ủy quyền:** Ứng dụng khách gửi yêu cầu tới máy chủ OAuth để xin quyền truy cập vào dữ liệu người dùng.
2. **Đăng nhập và đồng ý:** Người dùng được chuyển hướng tới trang đăng nhập của nhà cung cấp dịch vụ OAuth để xác nhận danh tính và đồng ý với các quyền truy cập.
3. **Nhận mã ủy quyền:** Nếu người dùng đồng ý, máy chủ sẽ gửi mã ủy quyền về cho ứng dụng khách qua một redirect URI.
4. **Yêu cầu token truy cập:** Ứng dụng khách sử dụng mã ủy quyền để yêu cầu token truy cập từ máy chủ.



5. **Nhận token truy cập:** Nếu mã ủy quyền hợp lệ, máy chủ sẽ trả về token truy cập mà ứng dụng khách có thể sử dụng để gọi API và lấy dữ liệu của người dùng.
6. **Gọi API:** Ứng dụng khách sử dụng token truy cập để gọi các API nhằm lấy dữ liệu người dùng.

IV. Token

Token là một đoạn mã được sinh ra ngẫu nhiên bởi Authorization server khi có yêu cầu được gửi đến từ Client.

Có 2 loại token:

- Access token
- Refresh token

4.1. Access token

Là một đoạn mã dùng để xác thực quyền truy cập, cho phép ứng dụng bên thứ 3 có thể truy cập vào những dữ liệu của người dùng trong một phạm vi nhất định mà nó cho phép. Token này được gửi bởi Client như một tham số được truyền vào *header* trong mỗi request khi cần truy cập đến tài nguyên trong Resource server.

Nếu để lộ mất *access token* thì cũng có thể coi như bị lộ *password* bởi có thể lợi dụng nó để lấy được những tài nguyên mà nó đang bảo vệ. Vì vậy, *access token* có một thời gian sử dụng nhất định (2 giờ, 2 tháng...) tùy thuộc vào nhu cầu sử dụng cũng như yêu cầu về tính bảo mật. *Access token* chỉ được sử dụng một lần duy nhất, khi nó hết hiệu lực Client sẽ phải gửi lại yêu cầu đến Authorization server để lấy một mã *access token* mới.

4.2. Refresh token



Được sinh ra bởi Authorization server, cùng lúc với *access token* nhưng lại khác nhau về chức năng. *Refresh token* sẽ được gửi đi để lấy về một *access token* mới khi nó hết hạn, cũng chính vì vậy nó có thời gian hiệu lực lâu hơn *access token*. Với *access token* thời gian hiệu lực có thể là 2 giờ thì *refresh token* có thể lên đến 10 giờ.

Việc có mặt của *refresh token* giúp cho Client có thể lấy lại được *access token* mà không cần phải nhận xác thực lại từ phía người dùng. Nếu người dùng đăng xuất, *refresh token* cũng sẽ bị xóa theo.

V. Các loại cấp phép (Grant Types)

5.1. Authorization Code Grant

Mô tả: Đây là loại cấp phép an toàn nhất, được sử dụng chủ yếu cho các ứng dụng web. Quy trình diễn ra theo các bước sau:

1. Ứng dụng khách chuyển hướng người dùng đến máy chủ OAuth với một yêu cầu bao gồm các tham số cần thiết (*client_id*, *redirect_uri*, *scope*, và *state*).
2. Người dùng đăng nhập và đồng ý cấp quyền cho ứng dụng khách.
3. Máy chủ OAuth chuyển hướng người dùng trở lại ứng dụng khách với một mã ủy quyền (*authorization code*) qua *redirect_uri*.
4. Ứng dụng khách gửi mã ủy quyền và thông tin xác thực đến máy chủ OAuth để đổi lấy token truy cập.

Ví dụ

1. Chuyển hướng người dùng để yêu cầu ủy quyền

```
const redirectUri = "https://yourapp.com/callback";
```

```
const authorizationUrl =  
`https://oauth-provider.com/auth?response_type=code&client_id=YOUR_CLIENT_ID&redirect_uri=${redirectUri}&scope=read write&state=random_state_string`;
```

```
window.location.href = authorizationUrl;
```

2. Callback handler để nhận mã ủy quyền



```
app.get('/callback', (req, res) => {  
  const { code, state } = req.query;  
  
  // Kiểm tra tham số state để bảo vệ chống CSRF  
  if (state !== expectedState) {  
    return res.status(403).send('Invalid state parameter');  
  }  
}
```

3. Đổi mã ủy quyền lấy token truy cập

```
const tokenUrl = 'https://oauth-provider.com/token';  
axios.post(tokenUrl, {  
  grant_type: 'authorization_code',  
  code: code,  
  redirect_uri: redirectUri,  
  client_id: 'YOUR_CLIENT_ID',  
  client_secret: 'YOUR_CLIENT_SECRET'  
})  
  .then(response => {  
    const accessToken = response.data.access_token;  
    // Sử dụng token truy cập  
  })  
  .catch(error => {  
    console.error('Error fetching access token:', error);  
  });  
});
```



Ưu điểm:

- An toàn hơn vì mã ủy quyền không được truyền trực tiếp qua URL và chỉ có thể được sử dụng một lần.
- Hỗ trợ việc lưu trữ token truy cập trên server, giúp tăng cường bảo mật.

Nhược điểm:

- Quy trình phức tạp hơn, yêu cầu nhiều bước.

5.2. Implicit Grant

Mô tả: Phương pháp này được thiết kế cho các ứng dụng web phía client (SPA) mà không có backend để bảo vệ thông tin nhạy cảm. Token truy cập được cấp trực tiếp mà không cần mã ủy quyền.

Quy trình:

1. Ứng dụng khách gửi yêu cầu tới máy chủ OAuth với tham số `response_type=token`.
2. Người dùng đăng nhập và đồng ý cấp quyền.
3. Máy chủ OAuth gửi token truy cập ngay lập tức qua URL fragment.

Ví dụ

1. Chuyển hướng người dùng để yêu cầu ủy quyền với Implicit Grant

```
const redirectUri = "https://yourapp.com/callback";
```

```
const authorizationUrl =  
`https://oauth-provider.com/auth?response_type=token&client_id=YOUR_CLIENT_ID&redirect_uri=${redirectUri}&scope=read write&state=random_state_string`;
```

```
window.location.href = authorizationUrl;
```

2. Nhận token truy cập trong callback

```
app.get('/callback', (req, res) => {
```



```
// Token truy cập được gửi qua URL fragment
const accessToken = req.fragment.access_token;

// Sử dụng token truy cập
});
```

Ưu điểm:

- Đơn giản và nhanh chóng, dễ dàng cho người dùng.
- Không yêu cầu bước thứ hai để đổi mã ủy quyền lấy token.

Nhược điểm:

- Token truy cập dễ bị lộ nếu URL không được bảo vệ.
- Thiếu khả năng bảo mật, không phù hợp cho ứng dụng nhạy cảm.

5.3. Resource Owner Password Credentials Grant

Mô tả: Phương pháp này được sử dụng khi người dùng tin tưởng ứng dụng khách và cung cấp trực tiếp thông tin đăng nhập của họ. Ứng dụng khách sẽ nhận được token truy cập bằng cách gửi thông tin đăng nhập của người dùng.

Quy trình:

1. Người dùng cung cấp tên người dùng và mật khẩu cho ứng dụng khách.
2. Ứng dụng khách gửi yêu cầu đến máy chủ OAuth với thông tin đăng nhập.
3. Máy chủ xác thực thông tin và trả về token truy cập.

Ví dụ

1. Người dùng cung cấp thông tin đăng nhập

```
const username = 'user@example.com';
const password = 'password123';
```



```
axios.post('https://oauth-provider.com/token', {  
  grant_type: 'password',  
  username: username,  
  password: password,  
  client_id: 'YOUR_CLIENT_ID',  
  client_secret: 'YOUR_CLIENT_SECRET'  
})  
  
.then(response => {  
  const accessToken = response.data.access_token;  
  // Sử dụng token truy cập  
})  
  
.catch(error => {  
  console.error('Error fetching access token:', error);  
});
```

Ưu điểm:

- Đơn giản cho người dùng, không cần phải thực hiện nhiều bước.
- Phù hợp cho ứng dụng nội bộ hoặc trong môi trường kiểm soát.

Nhược điểm:

- Không an toàn, vì thông tin đăng nhập của người dùng dễ bị lộ.
- Không phù hợp cho các ứng dụng bên thứ ba.

5.4. Client Credentials Grant

Mô tả: Phương pháp này được sử dụng cho các ứng dụng server-to-server, nơi không có người dùng. Ứng dụng khách gửi thông tin xác thực của mình để nhận token truy cập.



Quy trình:

1. Ứng dụng khách gửi yêu cầu đến máy chủ OAuth với client ID và client secret.
2. Máy chủ xác thực thông tin và trả về token truy cập.

Ví dụ

1. Gửi yêu cầu lấy token truy cập cho ứng dụng không có người dùng

```
axios.post('https://oauth-provider.com/token', {  
  grant_type: 'client_credentials',  
  client_id: 'YOUR_CLIENT_ID',  
  client_secret: 'YOUR_CLIENT_SECRET'  
})  
  
.then(response => {  
  const accessToken = response.data.access_token;  
  // Sử dụng token truy cập  
})  
  
.catch(error => {  
  console.error('Error fetching access token:', error);  
});
```

Ưu điểm:

- Phù hợp cho các ứng dụng server-to-server.
- Không yêu cầu thông tin người dùng, giúp đơn giản hóa quy trình.

Nhược điểm:

- Chỉ có thể sử dụng cho các ứng dụng mà không cần thông tin người dùng.



VI. Các lỗ hổng trong xác thực OAuth

Dưới đây là ba lỗ hổng chính trong OAuth cùng với ví dụ cụ thể để bypass:

6.1. Thiếu cấu hình bảo mật

Nếu các cài đặt bảo mật không được cấu hình đúng, kẻ tấn công có thể dễ dàng lấy token truy cập hoặc mã ủy quyền.

Ví dụ

Khách hàng không kiểm tra `redirect_uri`

```
const redirectUri = "http://malicious-site.com"; // Một URL độc hại
const authorizationUrl =
`https://oauth-provider.com/auth?response_type=code&client_id=YOUR_CLIENT_ID&redirect_uri=${redirectUri}`;
window.location.href = authorizationUrl;
```

6.2. Xác thực kém

Nếu không có các biện pháp xác thực mạnh mẽ, kẻ tấn công có thể sử dụng mã ủy quyền mà không cần sự đồng ý của người dùng.

Ví dụ

Giả sử mã ủy quyền đã bị lộ

```
curl -X POST https://oauth-provider.com/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d
"grant_type=authorization_code&code=YOUR_AUTH_CODE&redirect_uri=https://yourapp.com/callback&client_id=YOUR_CLIENT_ID&client_secret=YOUR_CLIENT_SECRET"
```

6.3. Tấn công CSRF (Cross-Site Request Forgery)

Nếu tham số `state` không được sử dụng hoặc xác thực, một kẻ tấn công có thể thực hiện một cuộc tấn công CSRF.

Ví dụ



Khi gửi yêu cầu ủy quyền

```
const state = Math.random().toString(36).substring(2); // Tạo state ngẫu nhiên
const redirectUri = "https://yourapp.com/callback";
const authorizationUrl =
`https://oauth-provider.com/auth?response_type=code&client_id=YOUR_CLIENT_ID&redirect_uri=${redirectUri}&state=${state}`;
window.location.href = authorizationUrl;
Callback handler

app.get('/callback', (req, res) => {
  const { code, state } = req.query;

  // Kiểm tra trạng thái để bảo vệ chống CSRF
  if (state !== expectedState) {
    return res.status(403).send('Invalid state parameter');
  }
  // Xử lý mã ủy quyền...
});
```

VII. Bảo vệ ứng dụng chống lại các lỗ hổng OAuth

Để bảo vệ ứng dụng khỏi các lỗ hổng này, hãy thực hiện các biện pháp sau:

- **Cấu hình bảo mật đúng:** Đảm bảo rằng các cài đặt bảo mật, như `redirect_uri`, được kiểm tra kỹ lưỡng.
- **Sử dụng xác thực mạnh:** Áp dụng các biện pháp xác thực mạnh mẽ cho mã ủy quyền và token truy cập, chẳng hạn như mã hóa và thời gian hết hạn.
- **Sử dụng tham số state:** Đảm bảo rằng tham số `state` được sử dụng trong tất cả các yêu cầu ủy quyền để ngăn chặn CSRF.
- **Giám sát và phát hiện:** Sử dụng các công cụ giám sát để phát hiện hành vi đáng ngờ liên quan đến xác thực và quyền truy cập.

VIII. Tham khảo

<https://portswigger.net/web-security/oauth#what-is-oauth>

<https://viblo.asia/p/tim-hieu-doi-chut-ve-oauth2-eW65GvMLlDO>

<https://auth0.com/docs>

<https://oauth.net/2/>



Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team